UE03

Methods & Combat

LEGAL NAME	*LEONARD-ADRIAN BUNEA*
PREFERRED NAME	ZOE BUNEA
TIME SPENT	31:29:49 *(too much)*

Table of Contents

Table of Figures

excidium

I had a bit too much fun with this exercise and created an entire playable text-based RPG game! I'm calling this excidium, Latin for destruction or demise. In this chapter I want to make a small overview of what excidium is and how to play it.

```
                    .--    .---.--
  ____ ___  ___ ____ |__| __| _/|__|__ __  _____
_/ __ \\  \/  // ___\|  |/ __ | |  |  |  \/     \
\  ___/ >    <\  \___|  / /_/ | |  |  |  /  Y Y  \
 \___  >__/\_ \\___  >__\____ | |__|____/|__|_|  /
     \/      \/    \/        \/                \/


   by Zoe McFife
```

Figure 1: excidium Logo

Start of the Game

```
                    .--    .---.--
  ____ ___  ___ ____ |__| __| _/|__|__ __  _____
_/ __ \\  \/  // ___\|  |/ __ | |  |  |  \/     \
\  ___/ >    <\  \___|  / /_/ | |  |  |  /  Y Y  \
 \___  >__/\_ \\___  >__\____ | |__|____/|__|_|  /
     \/      \/    \/        \/                \/


   by Zoe McFife

   1. Create Character
   2. Choose Difficulty
   3. Start Game
   4. Exit
Choose an option (1-4)
```

Figure 2: excidium main menu

To start excidium the player must first create their character and choose a difficulty.

Character Creation

```
=== Character Creator ===


Enter character name: Starjumper

Allocate your skill points:
1. Strength: 1
2. Dexterity: 1
3. Intelligence: 1
Remaining Points: 5
Choose a stat to increase: (1=STR, 2=DEX, 3=INT)
>
```

The character creator is simple. The player must name their character and has five skill points available to add to their character's strength, dexterity and intelligence.

Strength increases damage of physical weapons and increases carry capacity; dexterity determined dodge change and turn order and intelligence influences the damage of magical weapons. All these also increase the players' max health. Stats will be covered in more detail later.

After the stat points have been distributed, the player gets prompted, whether they are satisfied with their character or not.

```
=== Starjumper ===


== Stats ==

Health: 182.0 / 182.0
Strength:     5
Dexterity:    2
Intelligence: 1
Status: ALIVE

== Equipment ==

Weapon: Fists +14.0 P DMG +16.0
Name:   Fists
```

```
Weight:  0.0
Value:   0.0
Rarity:  LOW
Damage:  14.0
Is Magic:      false
Special: Desperate Flurry
Special Damage:      16.0
Armor:   Clothes +7.0 DEF (PRISTINE)
Name:    Clothes
Weight:  1.0
Value:   50.0
Rarity:  LOW
Defense: 7.0
Durability:    675.0 / 675.0 (PRISTINE)
State:   PRISTINE
Shield:  Fists +4.0 DEF
Name:    Fists
Weight:  0.0
Value:   0.0
Rarity:  LOW
Defense: 4.0

Are you satisfied with your creation? (y/n):
```

Figure 3: Character creation satisfaction prompt

Difficulty Selection

excidium has three difficulties: easy, medium and hard. These determine how fast more difficult battles appear. I don't recommend playing with hard difficulty.

```
=== Difficulty Selection ===


Difficulty:
1. EASY: Smell the flowers.
2. MEDIUM: Hold your ground.
3. HARD: This will make your life hell.
Choose a stat to increase: (1=EASY, 2=MEDIUM, 3=HARD)
>
```

Figure 4: Difficulty Selection

Playing excidium

After having created a character and selected a difficulty, you can select the start option. A battle will start. Battles can have up to four enemies!

Battles

```
  ____         _    _   _          _____ _              _
 |  _ \       | |  | | | |        / ____| |            | |
 | |_) | __ _| |_ | |_| | ___   | (___ | |_ __ _ _ __| |_
 |  _ < / _` | __| __| |/ _ \  \___ \| __/ _` | '__| __|
 | |_) | (_| | |_| |_| |  __/  ____) | || (_| | |  | |_
 |____/ \__,_|\__|\__|_|\___| |_____/ \__\__,_|_|   \__|


Battle 1 begins!
Danger Level: HARMLESS

=== Turn 1 ===

----------------------------------------
| Abyssal Marauder                     |
| Health: 35 / 35                      |
| ==================================== |
| D2, I1, S1                           |
| Attack: 14, Defense: 20              |
| (ALIVE)                              |
----------------------------------------
----------------------------------------
| Starjumper                           |
| Health: 182 / 182                    |
| ==================================== |
| Level 1 - D2, I1, S5                 |
| Attack: 18, Defense: 18              |
| Armour: PRISTINE                     |
| (ALIVE)                              |
----------------------------------------
```

Figure 5: Battle Start

Battles range from HARMLESS; MOSTLY HARMLESS, DANGEROUS, EXTREME and DEATH. Each of these difficulties have varied enemy encounters and different loot pools.

Turns

A turn is separated into multiple passes. Turn order is determined by each character's dexterity stat. The higher the dexterity, the sooner a character has its turn.

The first pass is the thinking pass. During the thinking pass, each character can determine their next action. Enemies can ATTACK, DEFEND and HEAL. The player can additionally USE ITEMs or PP ATTACK.

```
Abyssal Marauder is thinking...

=== Your Turn! ===


== Select an Enemy to Target ==

1. Abyssal Marauder
Select an enemy by entering the corresponding number.
> 1

== Battle Options ==

Current PP: 0/100
1. Attack
2. Desperate Flurry (5 PP)
3. Defend
4. Use Item
  Select an action by entering the corresponding number.
```

Figure 6: Thinking pass

An enemy thinking is displayed as "Enemy is thinking…". Once it's the players' turn to think, they get a battle options menu. What PP is and what PP attacks are will be explained later. Let's just defend against the Abyssal Marauder for now.

```
> 3
You chose to Defend!
```

```
Starjumper is defending for 4.0 extra defense!
Starjumper gained 10 PP! (Current PP: 20)
```

Figure 7: Defense Pass

After the thinking pass is the defense pass. A character that chooses to defend raises their shield during this pass and gets extra defense determined by the wielded shield. While defending, a player earns Power Points or PP for short.

Next up is the action pass. In this pass characters execute their ATTACK, PP ATTACK or USE ITEM.

```
Abyssal Marauder attacks Starjumper for 14 damage!
Starjumper takes 0 damage!
```

Figure 8: Action Pass

In this case, the Abyssal Marauder decided to attack us. We took no damage, because our defense was higher than the Abyssal Marauders attack.

```
Starjumper stopped defending!
```

Figure 9: Stop defense pass for cleanup

After the action pass, there's an additional pass that just lowers the shield of every currently defending character.

While defending we gathered enough PP to use our PP attack "Desperate Flurry". Every weapon has its own PP attack, that requires PP to use but deals a lot more damage.

```
Current PP: 10/100
1. Attack
2. Desperate Flurry (5 PP)
3. Defend
4. Use Item
Select an action by entering the corresponding number.
> 2
```

Figure 10: PP attack selection

```
Abyssal Marauder attacks Starjumper for 14 damage!
Starjumper takes 0 damage!
Desperate Flurry strikes with overwhelming force!
Starjumper uses Desperate Flurry on Abyssal Marauder for 34
damage!
PP remaining: 15
Abyssal Marauder takes 14 damage!
```

Figure 11. Desperate Flurry PP attack

Our PP attack did 14 damage. PP attacks are very powerful compared to regular attacks. For difficult enemies, it's a good strategy to defend and save PP until you can do multiple PP attacks after each other.

After doing enough damage to an enemy, they enter the CRITICALLY HURT status, which drastically reduces their ATTACK ability. Now is the perfect time to use a power powerful PP attack to kill the enemy.

```
----------------------------------------
| Abyssal Marauder                     |
| Health: 7 / 35                       |
| ======                               |
| D2, I1, S1                           |
| Attack: 4, Defense: 20               |
| (CRITICALLY_HURT)                    |
----------------------------------------
```

Figure 12: Critically hurt

```
Abyssal Marauder is defending for 7.0 extra defense!
Desperate Flurry strikes with overwhelming force!
Starjumper uses Desperate Flurry on Abyssal Marauder for 34
damage!
PP remaining: 5
Abyssal Marauder takes 7 damage!
Abyssal Marauder died!
Starjumper gained 50 experience points.
Starjumper leveled up to level 2!
```

Figure 13: Abyssal Marauder death

The Abyssal Marauder is now dead. The player gained 50 experience points and leveled up. The battle is now over.

```
Battle 1 has ended!
Press Enter to continue...
```

Figure 14: Battle 1 has ended!

After the battle is over, it's time to pick up loot!

Loot

```
.____    ________   ___________________
|    |   \_____  \  \_____  \__    ___/
|    |    /   |   \  /   |   \|    |
|    |___/    |    \/    |    \    |
|_______ \_______  /\_______  /____|
        \/       \/         \/
You can still loot 4 items. THEY ARE COMING...
Lootable Items:
Carry Capacity: 2/25.0
0: Exit
1: Minor Lifeforce Potion +40.0 HP | Weight: 1.2
2: Spore Spitter +13.0 P DMG +11.0 | Weight: 3.0    Compare: -
1.0 DMG
3: Rundown Vest +8.9 DEF (PRISTINE) | Weight: 3.5    Compare:
+1.9196841757531242 DEF
4: Cargo Clamp +7.0 DEF | Weight: 3.0    Compare: +3.0 DEF
Select an item to loot!
>
```

Figure 15: Loot screen

Every enemy carries a weapon, armor, a shield and healing items. All of these get dropped after the battle concludes. The player can select a couple items, less or more depending on the selected difficulty. Each item also displays its stats and a comparison with the currently equipped item.

Rest

```
 _____  ______  _____ _______
|  __ \|  ____|/ ____|__   __|
| |__) | |__  | (___    | |
|  _  /|  __|  \___ \   | |
| | \ \| |____ ____) |  | |
|_|  \_\______|_____/   |_|
```

```
----------------------------------------
| Starjumper                           |
| Health: 252 / 252                    |
| ==================================== |
| Level 2 - D2, I1, S5                 |
| Attack: 18, Defense: 18              |
| Armour: PRISTINE                     |
| (ALIVE)                              |
----------------------------------------

== You have survived the battle! ==

1. View Character
2. Use Stat Points (1 available)
3. Equip Item
4. Drop Item
5. Use Items
6. Continue Journey
7. Give up
Select an action by entering the corresponding number.
>
```

Figure 16: Rest menu

After looting, the player can rest. During resting the player has 7 options.

View Character

View Character displays the player's stats and inventory in detail.

```
=== Information ===

Name:          Starjumper
Level:         2
Experience:    0/100

=== Stats ===

Health:        252.0/252.0
Defense:             18.0
Attack:        18.2
Strength:            5
Dexterity:           2
```

```
Intelligence:       1

=== Equipment ===


== Weapon ==

Name:   Fists
Weight: 0.0
Value:  0.0
Rarity: LOW
Damage: 14.0
Is Magic:     false
Special: Desperate Flurry
Special Damage:     16.0

== Armor ==

Name:   Clothes
Weight: 1.0
Value:  50.0
Rarity: LOW
Defense: 7.0
Durability:   675.0 / 675.0 (PRISTINE)
State:  PRISTINE

== Shield ==

Name:   Fists
Weight: 0.0
Value:  0.0
Rarity: LOW
Defense: 4.0

=== Inventory ===

Inventory:
Weight: 9.9/25.0
1: Refreshing Decoction +38.0 HP | Weight: 1.2
2: Stitching Kit +25.0 HP | Weight: 0.9
3: Cute cat bandaid +25.0 HP | Weight: 0.1
```

```
4: Rundown Vest +8.9 DEF (PRISTINE) | Weight: 3.5    Compare:
+1.9 DEF
5: Cargo Clamp +7.0 DEF | Weight: 3.0    Compare: +3.0 DEF
6: Minor Lifeforce Potion +40.0 HP | Weight: 1.2
```

Figure 17: View Character

Use Stat Points

After leveling up, the player gains a spendable skill point.

```
Allocate your skill points:
1. Strength: 5
2. Dexterity: 2
3. Intelligence: 1
Remaining Points: 1
Choose a stat to increase: (1=STR, 2=DEX, 3=INT)
>
```

Figure 18: Allocate skill point

Item Actions

During rest the player can equip, drop or use items.

```
> 3
Inventory:
Weight: 9.9/30.0
1: Refreshing Decoction +38.0 HP | Weight: 1.2
2: Stitching Kit +25.0 HP | Weight: 0.9
3: Cute cat bandaid +25.0 HP | Weight: 0.1
4: Rundown Vest +8.9 DEF (PRISTINE) | Weight: 3.5    Compare:
+1.9 DEF
5: Cargo Clamp +7.0 DEF | Weight: 3.0    Compare: +3.0 DEF
6: Minor Lifeforce Potion +40.0 HP | Weight: 1.2
> 4
Starjumper obtained Clothes!
Starjumper equipped Rundown Vest!
```

Figure 19: Player equips Rundown Vest

Continue Journey

Choosing to continue the journey starts the next battle.

Give up

Giving up makes the player commit suicide.

```
You have chosen to give up. Game Over.
Starjumper has given up.
Starjumper gained 10 PP! (Current PP: 15)
Starjumper attacks Starjumper for 19 damage!
Starjumper takes 19 damage!

…

Starjumper gained 0 PP! (Current PP: 135)
Starjumper attacks Starjumper for 6 damage!
Starjumper takes 6 damage!
Starjumper died!

Starjumper has given up.
```

Death

```
                    .__    .___.__
  ____ ___  ___ ____ |__| __| _/|__|__ __  _____
_/ __ \\  \/  // ___\|  |/ __ | |  |  |  \/     \
\  ___/ >    <\  \___|  / /_/ | |  |  |  /  Y Y  \
 \___  >__/\_ \\___  >__\____ | |__|____/|__|_|  /
     \/      \/    \/        \/                \/


by Zoe McFife


=== Starjumper has perished in battle! ===

----------------------------------------
| Starjumper                           |
| Health: 0 / 270                      |
```

```
|                                       |
| Level 2 - D2, I1, S6                  |
| Attack: 19, Defense: 19               |
| Armour: SCRATCHED                     |
| (DEAD)                                |
-----------------------------------------
Press Enter to continue...
```

After dying, your character gets displayed one more time. You'll be forwarded back to the main menu and you'll need to create a new character.

GameCharacter

The GameCharacter class represents every character in the game. Due to the shear size of this class, only the most important portions will be covered in this document. Please refer to the source code or the included JavaDoc for all details.

```
private String name;
public ActionType nextAction;

private double health;
private double maxHealth;

private int strength;
private int dexterity;
private int intelligence;

public static final int MIN_STAT_VALUE = 1;

public static final int MAX_STAT_VALUE = 10;

private Weapon equippedWeapon =
WeaponFactory.createBaseWeapon();
private Shield equippedShield =
ShieldFactory.createBaseShield();
private Armour equippedArmour =
ArmourFactory.createBaseArmour();

private boolean isDefending = false;

private final Inventory inventory = new Inventory(this);
```

Figure 20: GameCharacter Fields

Combat Methods

GameCharacter handles all combat actions like attacking or defending.

Attacking and getting attacked

The attack method gets a GameCharacter as its target. It calls the the takeDamage() method on the target. If the GameCharacter is a player, it also calls the gainPP method.

```
/**
 * Attacks another character.
 * Does not attack if the target is already dead.
 *
 * @param target The character to attack
 */
public void attack(GameCharacter target)
{
    if (!target.isAlive())
    {
        return;
    }

    if (this instanceof Player player)
    {
        player.gainPP(equippedWeapon.getPpGainPerUse());
    }

    IO.println(getName() + " attacks " + target.getName() + "
for " + Math.round(getDamage()) + " damage!");
    target.takeDamage(getDamage());
}
```

Figure 21: attack method

The getDamage() method returns the damage the GameCharacter can do. Damage is determined by the equipped Weapon, the strength and the intelligence stat. Additionally a damage multiplier gets applied, if the character is critically hurt. The formula for damage is shown in Figure 22.

```
equippedWeapon.getDamage() * (1 + stat *
GameManager.DAMAGE_MULTIPLIER_PER_STAT) * damage_multiplier
```

Figure 22: damage formula

```
/**
 * Calculates the character's damage output.
 * Uses strength for physical weapons, intelligence for magical
weapons.
 *
 * @return Total damage the character can deal
 */
public double getDamage()
{
    double damage_multiplier = 1;

    if (getStatus() == CharacterStatus.CRITICALLY_HURT)
    {
        damage_multiplier =
GameManager.DAMAGE_REDUCTION_WHEN_CRITICAL_STATUS;
    }

    if (equippedWeapon.isMagic())
    {
        return equippedWeapon.getDamage() * (1 + intelligence *
GameManager.DAMAGE_MULTIPLIER_PER_INTELLIGENCE) *
damage_multiplier;
    }

    return equippedWeapon.getDamage() * (1 + strength *
GameManager.DAMAGE_MULTIPLIER_PER_STRENGTH) * damage_multiplier;
}
```

Figure 23: getDamage method

The takeDamage method handles incoming damage. First, the dodgeRoll method gets called, which determines a dodge based on the players dexterity. Incoming damage gets subtracted by the current defense. Armour gets its durability reduced by the damage. Current health gets reduced by the damage after the defense got subtracted. If the character reaches zero health, the abstract onDeath method gets called.

```
/**
 * Applies damage to the character.
 * Damage is reduced by defense, then subtracted from health.
 * Character dies if health reaches 0.
 *
 * @param damage The raw damage to apply
 */
public void takeDamage(double damage)
{
    if (dodgeRoll())
    {
        IO.println(getName() + " dodged the attack!");
        return;
    }

    double damageTaken = Math.max(damage - getCurrentDefense(),
0);
    IO.println(getName() + " takes " + Math.round(damageTaken) +
" damage!");

    getEquippedArmour().reduceDurability(damage -
getEquippedArmour().getDefense());

    health -= damageTaken;
    if (health < 0)
    {
        health = 0;
        IO.println(getName() + " died!");
        onDeath();
    }
}
```

Figure 24: takeDamage method

Defense

The defend method raises the shield of a character. This sets the isDefending flag to true. The defense only gets calculated in the getCurrentDefense method seen in Figure 26.

```
/**
 * Puts the character in a defensive stance.
 * While defending, the character gains additional defense from
```

```
their shield.
 */
public void defend()
{
    IO.println(getName() + " is defending for " +
equippedShield.getDefense() + " extra defense!");
    isDefending = true;

    // Grant PP to players when defending
    if (this instanceof Player player)
    {
        player.gainPP(equippedShield.getPpGain());
    }
}
```

Figure 25: defend Method

The getCurrentDefense method returns the sum of the defense value of the equipped armor and if the player is defending, also the equipped shield.

```
/**
 * Calculates the character's total defense value.
 * Includes shield defense only when actively defending.
 *
 * @return Total defense points
 */
public double getCurrentDefense()
{
    if (isDefending())
    {
        return (getEquippedArmour().getDefense() +
getEquippedShield().getDefense());
    }

    return getEquippedArmour().getDefense();
}
```

Figure 26: getCurrentDefense method

Inventory Management

All inventory management methods interface with the inventory class.

The equipItem method takes in an item and with the use of a switch it determines which slot to equip the item. The “Fists” weapon and shield are handled differently here, because those are the defaults if no weapon or shield are equipped. It would be weird to have your own fists in the inventory. This only must be done because fists are a weapon and shield.

The displayMessage and storePreviousItem flags are only used in specific circumstances like character creation.

```
/**
 * Equips an item to the appropriate slot.
 * Automatically adds currently equipped item back to inventory
if it has weight.
 *
 *
 * @param item The item to equip
 * @param displayMessage if equip message gets displayed or not
 */
public void equipItem(Item item, boolean displayMessage, boolean
storePreviousItem)
{
    switch (item)
    {
        case Weapon weapon ->
        {
            /* Since fists are Weapons, check if an item has
weight or not before being added to inventory, so that the
player can't have their fists in the inventory */
            if (!equippedWeapon.getName().equals("Fists") &&
storePreviousItem)
            {
                addItemToInventory(equippedWeapon,
displayMessage, false);
            }
            equippedWeapon = weapon;

            if (displayMessage)
            {
                IO.println(getName() + " equipped " +
item.getName() + "!");
            }
```

```
            inventory.removeItem(item);
        }
        case Shield shield ->
        {
            if (!equippedShield.getName().equals("Fists") &&
storePreviousItem)
            {
                addItemToInventory(equippedShield,
displayMessage, false);
            }
            equippedShield = shield;

            if (displayMessage)
            {
                IO.println(getName() + " equipped " +
item.getName() + "!");
            }

            inventory.removeItem(item);
        }
        case Armour armour ->
        {
            if (item.getWeight() != 0 && storePreviousItem)
            {
                addItemToInventory(equippedArmour,
displayMessage, false);
            }
            equippedArmour = armour;

            if (displayMessage)
            {
                IO.println(getName() + " equipped " +
item.getName() + "!");
            }

            inventory.removeItem(item);
        }
        case null, default ->
        {
            if (displayMessage)
            {
                assert item != null;
```

```
                IO.println("Cannot equip " + item.getName() +
"!");
            }
        }
    }
}
```

Figure 27: equipItem method

The dropItem method removes the selected item from the inventory and displays the drop message.

```
/**
 * Drops an item from inventory.
 *
 * @param item The item to drop
 */
public void dropItem(Item item)
{
    inventory.removeItem(item);
    IO.println(getName() + " dropped " + item.getName() + "!");
}
```

Figure 28: dropItem method

The useItem method only works on healing items. It adds the corresponding amount of health to the character.

```
/**
 * Uses an item from inventory.
 * Currently only supports HealingPotions.
 *
 * @param item The item to use
 */
public void useItem(Item item)
{
    if (item instanceof HealingPotion)
    {
        setHealth(getHealth() + ((HealingPotion)
item).getHealingAmount());
        IO.println(getName() + " used " + item.getName() + " and
healed for " + ((HealingPotion) item).getHealingAmount() + "
health!");
```

```
            inventory.removeItem(item);
        }
        else
        {
            IO.println("Cannot use " + item.getName() + "!");
        }
    }
```

Figure 29: useItem method

DisplayBoxes

The static getDisplayBox() and combineEnemyBoxes() methods are responsible for displaying the character boxes seen in the battles.

Player

The player extends the GameCharacter class. It adds level management and power points.

```
private int level = 1;
private int experience = 0;
private int availableStatPoints = 0;
private int currentPP = 0;
private int maxPP = 100;
public static double DEFAULT_PLAYER_MAX_HEALTH = 100.0;
public static int DEFAULT_PLAYER_MAX_PP = 100;
```

Figure 30: Player fields

Level Management

Levels get accumulated through experience, which enemies give upon death.

```
/**
 * Adds experience points to the player and handles level-ups.
 * Automatically levels up when enough experience is gained.
 *
 * @param exp The amount of experience to add
 */
public void addExperience(int exp)
{
```

```
        this.experience += exp;

        IO.println(getName() + " gained " + exp + " experience
    points.");

        while (this.experience >= getExperienceNeededForNextLevel())
        {
            this.experience -= getExperienceNeededForNextLevel();
            levelUp();
        }
    }
```

Figure 31: addExperience method

Experience required to level up to level N is determined with the 40 * N – 1 formula.

```
    /**
     * Calculates the experience points needed to reach the next
    level.
     * Formula: 40 * current level
     *
     * @return Experience points required for next level
     */
    public int getExperienceNeededForNextLevel()
    {
        return (40 * this.level);
    }
```

Figure 32: getExperienceNeededForNextLevel method

With every level up, the player gains a stat point. The spendStatPoint method gets called during the spend stat points menu to safely reduce available stat points.

```
    /**
     * Spends one stat point.
     * Decreases available stat points by 1, with a minimum of 0.
     */
    public void spendStatPoint()
    {
        this.availableStatPoints -= 1;
```

```
    if (this.availableStatPoints < 0)
    {
        this.availableStatPoints = 0;
    }
}
```

Figure 33: spendStatPoint method

When levelling up, the players max health increases. The player also gets healed with twice the gain they receive. During multiple runs of the game, I noticed that's its too hard if the player doesn't get healed during level ups, but fully healing the player like in many other RPGs makes the game too easy.

```
/**
 * Levels up the player character.
 * Increases level, grants stat points, increases max PP and
health, and fully heals the player.
 * Private method called automatically when enough experience is
gained.
 */
private void levelUp()
{
    this.level += 1;
    this.availableStatPoints +=
GameManager.STAT_POINTS_PER_LEVEL;
    setMaxPP(getMaxPP() +
GameManager.MAX_PP_INCREASE_PER_LEVEL);
    setMaxHealth(getMaxHealth() +
GameManager.HEALTH_INCREASE_PER_LEVEL);
    setHealth(getHealth() +
GameManager.HEALTH_INCREASE_PER_LEVEL * 2);

    IO.println(getName() + " leveled up to level " + this.level
+ "!");
}
```

Figure 34: levelUp methods

Power Points

The player has a maximum amount of PP they can hold. The maximum PP increases with the player's level. The gainPP method adds PP and it gets called during attacks and defensive actions.

```
/**
 * Adds PP to the player's current PP pool.
 *
 * @param amount Amount of PP to add (must be positive)
 */
public void gainPP(int amount)
{
    if (amount <= 0)
    {
        return;
    }

    if (this.currentPP + amount > this.maxPP)
    {
        amount = this.maxPP - this.currentPP;
    }

    this.currentPP += amount;
    IO.println(getName() + " gained " + amount + " PP! (Current
PP: " + currentPP + ")");
}
```

Figure 35: gainPP method

Enemy

Enemies extend the GameCharacter class and add enemy logic.

Experience Reward

When an Enemy dies, they add their experience reward to the player.

```
/**
 * Called when the enemy dies.
 * Awards experience points to the player.
 */
@Override
protected void onDeath()
{
```

```
        GameManager.getPlayer().addExperience(experienceReward);
    }
```

Figure 36: onDeath method

The think methods displays the thinking status and gets the next action.

```
    /**
     * Makes the enemy think about their next action.
     * Determines the best action based on the current battle state.
     *
     * @param attacker The character the enemy is fighting against
     */
    public void think(GameCharacter attacker)
    {
        IO.println(getName() + " is thinking...");
        nextAction = calculateNextAction(attacker);
    }
```

Figure 37: thinkMethod

The calculateNextAction method calculates what action the enemy should take depending on several factors. The more hurt the player is, the more likely the enemy is to attack. The more hurt the enemy is, the more likely the enemy is to defend or use a healing item. The randomWeightedAction methods returns ATTACK, DEFEND or HEAL randomly and weighted by the given weights.

```
    /**
     * Calculates the next action for the enemy based on battle
    conditions.
     * Uses AI logic that considers both the enemy's and attacker's
    health status.
     *
     * @param attacker The character the enemy is fighting against
     * @return The calculated action type (ATTACK, DEFEND, or HEAL)
     */
    public ActionType calculateNextAction(GameCharacter attacker)
    {
        CharacterStatus selfStatus = getStatus();
        CharacterStatus attackerStatus = attacker.getStatus();
```

```
    if (randomSuicideTrigger())
    {
        return ActionType.SUICIDE;
    }

    if (attackerStatus == CharacterStatus.CRITICALLY_HURT)
    {
        return randomWeightedAction(80, 20, 0);
    }
    else if (attackerStatus == CharacterStatus.SEVERELY_HURT)
    {
        return randomWeightedAction(90, 5, 5);
    }

    if (selfStatus == CharacterStatus.CRITICALLY_HURT)
    {
        return randomWeightedAction(5, 35, 65);

    }
    else if (selfStatus == CharacterStatus.SEVERELY_HURT)
    {
        return randomWeightedAction(40, 40, 20);
    }
    else if (selfStatus == CharacterStatus.HURT)
    {
        return randomWeightedAction(80, 10, 10);
    }
    else
    {
        return ActionType.ATTACK;
    }
}
```

Figure 38: calculateNextAction

GameManager

The GameManager class holds most constants and holds a reference to the player. This class is used for adjusting game balance.

```
private static Player player;
/** Flag indicating whether the player has been initialized */
public static boolean hasPlayerBeenInitialized = false;

/** Damage multiplier applied per point of strength for physical
weapons */
public static double DAMAGE_MULTIPLIER_PER_STRENGTH = 0.06;

/** Damage multiplier applied per point of intelligence for
magical weapons */
public static double DAMAGE_MULTIPLIER_PER_INTELLIGENCE = 0.2;

/** Dodge chance percentage gained per point of dexterity */
public static double DODGE_CHANCE_PER_DEXTERITY = 0.02;

/** Weight carrying capacity granted per point of strength */
public static final int CARRY_CAPACITY_PER_STRENGTH = 5;

/** Current game difficulty setting */
public static Difficulty difficulty = Difficulty.NONE;

/** Number of turns before difficulty increases on Easy mode */
public static int DIFFICULTY_INCREASE_AFTER_TURNS_EASY = 12;

/** Number of turns before difficulty increases on Medium mode
*/
public static int DIFFICULTY_INCREASE_AFTER_TURNS_MEDIUM = 6;

/** Number of turns before difficulty increases on Hard mode */
public static int DIFFICULTY_INCREASE_AFTER_TURNS_HARD = 3;

/** Maximum enemies per battle at HARMLESS danger level */
public static int MAX_ENEMIES_PER_BATTLE_HARMLESS = 1;

/** Maximum enemies per battle at MOSTLY_HARMLESS danger level
*/
```

```
public static int MAX_ENEMIES_PER_BATTLE_MOSTLY_HARMLESS = 1;

/** Maximum enemies per battle at DANGEROUS danger level */
public static int MAX_ENEMIES_PER_BATTLE_DANGEROUS = 2;

/** Maximum enemies per battle at EXTREME danger level */
public static int MAX_ENEMIES_PER_BATTLE_EXTREME = 3;

/** Maximum enemies per battle at DEATH danger level */
public static int MAX_ENEMIES_PER_BATTLE_DEATH = 4;

/** Number of items able to be looted after battle on Easy mode
*/
public static int ITEM_LOOT_COUNT_EASY = 8;
/** Number of items able to be looted after battle on Medium
mode */
public static int ITEM_LOOT_COUNT_MEDIUM = 5;
/** Number of items able to be looted after battle on Hard mode
*/
public static int ITEM_LOOT_COUNT_HARD = 3;

public static int MAX_PP_INCREASE_PER_LEVEL = 35;
public static int STAT_POINTS_PER_LEVEL = 1;
public static double HEALTH_INCREASE_PER_LEVEL = 70;
public static double HEALTH_INCREASE_PER_STRENGTH = 18;
public static double HEALTH_INCREASE_PER_INTELLIGENCE = 10;
public static double HEALTH_INCREASE_PER_DEXTERITY = 10;

public static double DAMAGE_REDUCTION_WHEN_CRITICAL_STATUS =
0.3;

public static double DELAY_SHORT = 0.5;
public static double DELAY_MEDIUM = 1;
public static double DELAY_LONG = 2;

public static double PLAYER_BASE_DEFENCE = 11;
```

Figure 39: GameManager Fields

Inventory

The Inventory is a class with an ArrayList and several helper methods to

```
private final List<Item> items = new ArrayList<>();
private final GameCharacter character
```

Figure 40: Inventory fields

Items

Items are all objects the player can pick up, use or equip. An item itself is generic and no use. All items have a weight, value and a rarity from low to legendary.

```
private String name;
private double weight;
private double value;
private ItemRarity rarity;
```

Figure 41: Item fields

Weapons

Weapons can be equipped. They determine attack damage and the special PP attack. Weapons also produce PP when used. The isMagic flag gets used to determine whether intelligence or strength gets used for the bonus damage.

```
private double damage;
private boolean isMagic;
private double specialDamage;
private String specialAttackName;
private String specialFlavorText;
private int ppCost;
private int ppGainPerUse;
```

Figure 42: Weapon fields

Shields

Shields have a defense value and ppGain. As of time of writing, these also get displayed during turns and in the item name.

```
private double defense;
private int ppGain;
```

Figure 43: Shield fields

Armours

Armours are special. They are the only items that can degrade. The durability of a piece of armour determines how much damage they can block before being breaking.

```
/** Default maximum durability for armour when not specified */
private static final double DEFAULT_MAX_DURABILITY = 100.0;

private double defense;
/**
 * The durability of the armour. When durability reaches 0, the
armour breaks and provides no defense.
 */
private double durability = DEFAULT_MAX_DURABILITY;
/**
 * The maximum durability of the armour.
 */
private double maxDurability = DEFAULT_MAX_DURABILITY;
```

Figure 44: Armour fields

Armours have different states depending on durability: PRISTINE, SCRATCHED, WORN, DAMAGED and BROKEN. These affect how fast the armour gets used up.

```
/**
 * Gets the current state of the armour based on its durability
percentage.
 * States range from PRISTINE (>= 95%) to BROKEN (< 10%).
 *
 * @return The current armour state
 */
```

```
public ArmourState getState()
{
    double durabilityRatio = durability / maxDurability;

    if (durabilityRatio >= 0.95) return ArmourState.PRISTINE;
    if (durabilityRatio >= 0.80) return ArmourState.SCRATCHED;
    if (durabilityRatio >= 0.50) return ArmourState.WORN;
    if (durabilityRatio >= 0.10) return ArmourState.DAMAGED;
    return ArmourState.BROKEN;
}
```

Figure 45: armour getState method

Each armour state has a wear multiplier. The more worn a piece of armour is, the faster it wears out.

```
/**
 * Reduces the durability of the armour by a specified amount,
adjusted by the armour's current state.
 *
 * @param amount The base amount to reduce durability by
 */
public void reduceDurability(double amount)
{
    if (amount < 0)
    {
        amount = 0;
    }

    ArmourState state = getState();
    double wearMultiplier = state.wearMultiplier;

    double finalWear = amount * wearMultiplier;

    durability = Math.max(0, durability - finalWear);
}
```

Figure 46: reduceDurability method

Healing Items

Healing items are referred to as Healing Potions in the code. They only have a healingAmount.

```
/** The amount of health this potion restores when used */
public double healingAmount;
```

Figure 47: Potion fields

Factories

Factories are classes that produce item and enemy instances. In general, each factory has:

- A static data class that holds data of the to be produced class instance
- Several hash maps and arrays that are presorted or linked with an ID or rarity.
- A static block that registers all new instances into the arrays
- Methods that return an instance based on a criteria

```
registerEnemy(4, "Fargoth Enforcer", 250, 8, 7, 6, "Heavy-hitter
in Fargoth armour", DangerLevel.EXTREME, 350);

registerArmour(101, "Satanic Loincloth", 3.0, 500, 30, 3000,
ItemRarity.MEDIUM);

registerWeapon(101, "PP Blade", 5, 15400000, 130, false,
ItemRarity.LEGENDARY, 160, "PP Special", "The PP surges through
your veins.", 100, 27);

registerShield(62, "Boneflame Guard", 8.0, 820, 27,
ItemRarity.HIGH, 35);
```

Figure 48: Example Registers

An example of a method a factory provides is seen in Figure 49. This method returns an enemy of a certain danger level.

```
/**
 * Creates a random enemy of the specified danger level.
 *
 * @param dangerLevel The desired danger level
 * @return A new random enemy of the specified danger level
 * @throws IllegalArgumentException if no enemies exist for the
given danger level
 */
public static Enemy createRandomEnemyByDangerLevel(DangerLevel
dangerLevel)
{
    List<EnemyData> enemies =
ENEMIES_BY_DANGER.get(dangerLevel);
    if (enemies == null || enemies.isEmpty())
    {
        throw new IllegalArgumentException("No enemies found for
danger level: " + dangerLevel);
    }
    EnemyData data =
enemies.get(random.nextInt(enemies.size()));
    return createEnemyFromData(data);
}
```

Figure 49: createRandomEnemyByDangerLevel example method

Battles

Battle

The Battle class holds every character that is currently participating in the battle.

```
private List<Enemy> enemies = new ArrayList<>();
private final List<GameCharacter> participantsOrderedByDexterity
= new ArrayList<>();
```

Figure 50: Battle fields

BattleGenerator

The BattleGenerator class is responsible for generating battles from a given danger level.

```
/**
 * Creates a new battle with enemies appropriate for the
specified danger level.
 *
 * @param dangerLevel The danger level determining enemy count
and strength
 * @return A new battle instance with generated enemies
 */
public static Battle generateBattle(DangerLevel dangerLevel)
{
    int numberOfEnemies =
getEnemyCountForDangerLevel(dangerLevel);

    List<Enemy> enemies = new ArrayList<>();

    for (int i = 0; i < numberOfEnemies; i++)
    {
        Enemy enemy = EnemyGenerator.generateEnemy(dangerLevel);
        enemies.add(enemy);
    }

    return new Battle(enemies, GameManager.getPlayer());
}
```

Figure 51: generateBattle method

EnemyGenerator

The EnemyGenerator class generates an enemy from a given danger level and equips the enemy with items corresponding to the danger level.

```
/**
 * Creates a fully equipped enemy with appropriate gear for the
danger level.
 *
 * @param dangerLevel The danger level determining enemy type
and equipment quality
 * @return A fully equipped enemy character
 */
public static Enemy generateEnemy(DangerLevel dangerLevel)
{
    Enemy enemy =
EnemyFactory.createRandomEnemyByDangerLevel(dangerLevel);
```

```
    Weapon weapon = generateWeapon(dangerLevel);
    Armour armour = generateArmour(dangerLevel);
    Shield shield = generateShield(dangerLevel);

    enemy.equipItem(weapon, false, false);
    enemy.equipItem(armour, false, false);
    enemy.equipItem(shield, false, false);

    List<HealingPotion> healingPotions =
generateHealingPotions(dangerLevel);

    for (HealingPotion potion : healingPotions)
    {
        enemy.addItemToInventory(potion, false);
    }

    return enemy;
}
```

Figure 52: generateEnemy method

User Interface

Every UI element extends the abstract UserInterface class that has the startUI method.

```
/**
 * Abstract base class for all user interface screens in the
game.
 * Defines the contract that all UI screens must implement.
 */
public abstract class UserInterface
{
    /**
     * Starts and displays this UI screen.
     * Each implementation should handle its own display logic
and user interaction.
     */
    public abstract void startUI();

}
```

Figure 53: UserInterface class

Since most UI elements are very similar in nature, I will only cover the most important ones.

UIHelper

The UIHelper utility class has methods for common UI actions like getting player inputs and displaying headings.

A very commonly used method is the getIntInput method. It safely gets player input for a selection.

```
/**
 * Gets an integer input from the player within a specified
range.
 * Continues to prompt until valid input is provided.
 *
 * @param min The minimum allowed value (inclusive)
 * @param max The maximum allowed value (inclusive)
 * @return The validated integer input
```

```
 */
public static int getIntInput(int min, int max)
{
    while (true)
    {
        try
        {
            IO.print("> ");

            int value = Integer.parseInt(IO.readln());

            if (value >= min && value <= max)
            {
                return value;
            }
        }
        catch (NumberFormatException ignored)
        {

        }

        IO.println("Invalid input. Enter a number between " +
min + " and " + max + ".");
    }
}
```

Figure 54: getIntInput method

Other often used methods are the wait for enter and delay methods.

```
/**
 * Pauses execution until the player presses Enter.
 */
public static void waitForEnterKey()
{
    IO.readln("Press Enter to continue...");
}
```

Figure 55: waitForEnterKey method

```
/**
 * Pauses execution for a short duration based on
```

```
GameManager.DELAY_SHORT.
 */

public static void delayShort()
{
    delay(GameManager.DELAY_SHORT);
}
```

Figure 56: delayShort method

Main Menu

The main menu is a good example of a UI component in excidium.

Most UI components follow a similar structure. A while loop with a specific condition, then some extra display methods to display menus and information. Then the player gets prompted to choose, and inside a switch statement, other UI components get started based on the player's choice. The UI components themselves run until they reach their own end condition.

```
@Override
public void startUI()
{
    while(true)
    {
        UIHelper.displayLogo();
        displayStartOptions();
        int choice = UIHelper.getIntInput(1, 4);

        switch (choice)
        {
            case 1:
                CharacterCreator characterCreator = new
CharacterCreator();
                characterCreator.startUI();

GameManager.setPlayer(characterCreator.getPlayerCharacter());
                break;
            case 2:
                DifficultySelection difficultySelection = new
DifficultySelection();
```

```
                difficultySelection.startUI();
                GameManager.difficulty =
difficultySelection.getSelectedDifficulty();
                break;
            case 3:


                if (!canGameStart())
                {
                    IO.println("You must create a character and
choose a difficulty before starting the game.");
                    UIHelper.waitForEnterKey();
                    break;
                }

                IO.println("Starting game...");
                UIHelper.clearScreen();

                GameLoop gameLoop = new GameLoop();
                gameLoop.startUI();

                break;
            case 4:
                IO.println("Exiting game. Goodbye!");
                System.exit(0);
                break;
            default:
                IO.println("Invalid choice. Please try again.");
                startUI();
                break;
        }

        UIHelper.clearScreen();
    }
}
```

Figure 57: Main menu startUI

Game Loop

The game loop UI component handles the entire game. It's simply a while loop that runs until the player is dead. Every iteration a battle is generated and started, then the rest UI gets started and the loop repeats. After the while loop ends, the deathUI gets started.

```
public void startUI()
{
    while (GameManager.getPlayer().isAlive())
    {
        UIHelper.clearScreen();

        BattleUI battleUI = new BattleUI(currentDangerLevel,
battleCount);
        battleUI.startUI();

        battleCount++;
        updateDangerLevel();

        UIHelper.clearScreen();

        if (GameManager.getPlayer().isAlive())
        {
            RestUI restUI = new RestUI();
            restUI.startUI();
        }
    }

    UIHelper.clearScreen();
    DeathUI deathUI = new DeathUI();
    deathUI.startUI();
}
```

Figure 58: Game Loop

BattleUI

The BattleUI generates a random battle and then starts the TurnUI. The TurnUI class handles the turn-based logic. After a battle has been cleared, it starts the looting UI component.

```
@Override
public void startUI()
{
    Battle battle = BattleGenerator.generateBattle(dangerLevel);

    displayBattleStartMessage();

    while (GameManager.getPlayer().isAlive() &&
!battle.isBattleOver())
    {
        TurnUI turnUI = new TurnUI(battle);
        turnUI.startUI();

        UIHelper.delayLong();

        UIHelper.clearScreen();

        displayBattleEndMessage();

        UIHelper.waitForEnterKey();
    }

    if (GameManager.getPlayer().isAlive())
    {
        UIHelper.clearScreen();
        LootUI lootUI = new LootUI(battle.getAllLoot());
        lootUI.startUI();
        UIHelper.waitForEnterKey();
    }
}
```

Figure 59: Battle UI

TurnUI

As explained in the first chapter, turns are separated into passes. The thinking pass calls the think method on enemies and prompts the player to select their next action.

```
@Override
public void startUI()
{
    while (GameManager.getPlayer().isAlive() &&
!battle.isBattleOver())
    {
        displayTurnHeader();

        UIHelper.displayEnemies(battle);
        UIHelper.displayPlayer(GameManager.getPlayer());

        thinkingPass();

        IO.println();

        defensePass();

        IO.println();

        actionPass();

        IO.println();

        stopDefendingPass();

        UIHelper.waitForEnterKey();
        UIHelper.clearScreen();
    }

}
```

Figure 60: Turn UI Start

```
/**
 * Thinking phase where each character decides their action.
 * Enemies use AI, player chooses through menu.
 */
private void thinkingPass()
{
    for (GameCharacter character :
battle.getParticipantsOrderedByDexterity())
    {
        if (character.isAlive())
        {
            if (character instanceof Enemy enemy)
            {
                enemy.think(GameManager.getPlayer());
                UIHelper.delayMedium();
            }
            else if (character instanceof Player)
            {
                UIHelper.printHeading("Your Turn!");

                enemySelection();
                actionSelection();

                IO.println();

                UIHelper.delayMedium();
            }
        }
    }
}
```

Figure 61: Thinking pass

```
/**
 * Applies defensive stance to characters who chose to defend.
 */
private void defensePass()
{
    for (GameCharacter character :
battle.getParticipantsOrderedByDexterity())
    {
```

```
        if (character.nextAction == ActionType.DEFEND &&
character.isAlive())
        {
            character.defend();
            UIHelper.delayMedium();
        }
    }
}
```

Figure 62: defense pass

```
    /**
     * Executes all character actions in turn order.
     * Characters attack or use healing items based on their
selected action.
     */
    private void actionPass()
    {
        for (GameCharacter character :
battle.getParticipantsOrderedByDexterity())
        {
            if (character.isAlive())
            {
                if (character.nextAction == ActionType.SUICIDE)
                {
                    character.suicide();
                }
                else if (character instanceof Enemy enemy)
                {
                    switch (enemy.nextAction)
                    {
                        case ATTACK ->
enemy.attack(GameManager.getPlayer());
                        case USE_ITEM -> enemy.useHealingItem();
                    }

                    UIHelper.delayMedium();
                }
                else if (character instanceof Player player)
                {
                    switch (player.nextAction)
```

```
                    {
                        case ATTACK ->
player.attack(battle.getEnemies().get(selectedEnemy - 1));
                        case USE_ITEM ->
                        {
                            ItemActionSelectionUI
itemActionSelectionUI = new ItemActionSelectionUI();
                            itemActionSelectionUI.startUI();
                        }
                        case USE_SPECIAL ->
player.useSpecial(battle.getEnemies().get(selectedEnemy - 1));
                    }
                    UIHelper.delayMedium();
                }
            }
        }
    }
```

Figure 63: Action pass

This pass is only for cleanup and I don't consider it to be one of the main passes.

```
    /**
     * Removes defensive stance from all defending characters at the
    end of turn.
     */
    private void stopDefendingPass()
    {
        for (GameCharacter character :
    battle.getParticipantsOrderedByDexterity())
        {
            if (character.isAlive())
            {
                if (character.isDefending())
                {
                    character.stopDefending();
                }
            }
        }
    }
```

Figure 64: stop defending pass

Error Handling Example

```
1. Create Character
2. Choose Difficulty
3. Start Game
4. Exit
Choose an option (1-4)
> dglogklfdhghdfkl
Invalid input. Enter a number between 1 and 4.
>
```

Figure 65: Error Handling Example

Example Run

A full unedited example run of excidium.

```
                    .__    .___.__
  ____ ___  ___ ____ |__| __| _/|__|__ __  _____
_/ __ \\  \/  // ___\|  |/ __ | |  |  |  \/     \
\  ___/ >    <\  \___|  / /_/ | |  |  |  /  Y Y  \
 \___  >__/\_ \\___  >__\____ | |__|____/|__|_|  /
     \/      \/    \/        \/                \/


by Zoe McFife

1. Create Character
2. Choose Difficulty
3. Start Game
4. Exit
Choose an option (1-4)
> 1
```

```
=== Character Creator ===


Enter character name: Starjumper

Allocate your skill points:
1. Strength: 1
2. Dexterity: 1
3. Intelligence: 1
Remaining Points: 5
Choose a stat to increase: (1=STR, 2=DEX, 3=INT)
> 1
```

```
Allocate your skill points:
1. Strength: 2
2. Dexterity: 1
3. Intelligence: 1
Remaining Points: 4
Choose a stat to increase: (1=STR, 2=DEX, 3=INT)
> 1
```

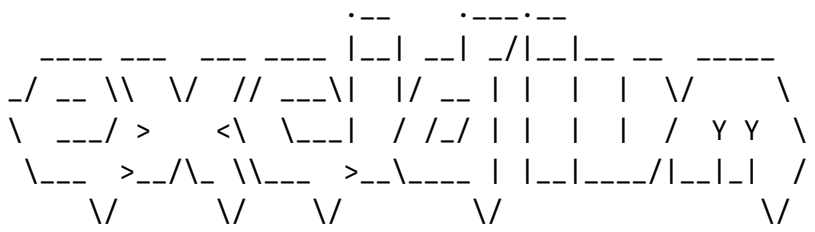
```
Allocate your skill points:
1. Strength: 3
2. Dexterity: 1
```

```
3. Intelligence: 1
Remaining Points: 3
Choose a stat to increase: (1=STR, 2=DEX, 3=INT)
> 2
```

```
Allocate your skill points:
1. Strength: 3
2. Dexterity: 2
3. Intelligence: 1
Remaining Points: 2
Choose a stat to increase: (1=STR, 2=DEX, 3=INT)
> 1
```

```
Allocate your skill points:
1. Strength: 4
2. Dexterity: 2
3. Intelligence: 1
Remaining Points: 1
Choose a stat to increase: (1=STR, 2=DEX, 3=INT)
> 1
```

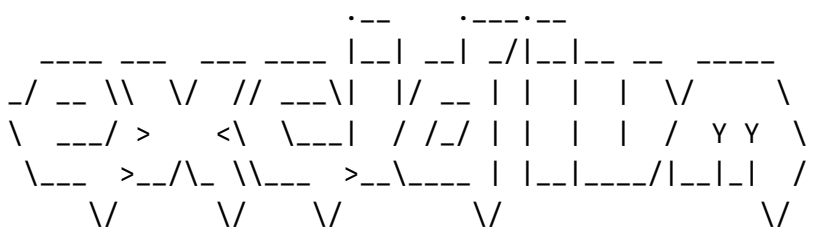
```
=== Starjumper ===


== Stats ==

Health: 182.0 / 182.0
Strength:     5
Dexterity:    2
Intelligence: 1
Status: ALIVE

== Equipment ==

Weapon: Fists +14.0 P DMG +16.0
Name:   Fists
Weight: 0.0
Value:  0.0
Rarity: LOW
Damage: 14.0
Is Magic:     false
Special: Desperate Flurry
Special Damage:     16.0
Armor:  Clothes +7.0 DEF (PRISTINE)
Name:   Clothes
Weight: 1.0
Value:  50.0
Rarity: LOW
Defense: 7.0
Durability:   675.0 / 675.0 (PRISTINE)
State:  PRISTINE
Shield: Fists +4.0 DEF
Name:   Fists
Weight: 0.0
Value:  0.0
Rarity: LOW
Defense: 4.0

Are you satisfied with your creation? (y/n): y
```



by Zoe McFife

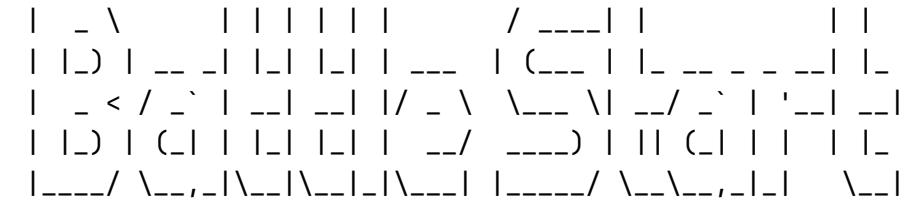
1. Create Character (Current: Starjumper)
2. Choose Difficulty
3. Start Game
4. Exit
Choose an option (1-4)
> 2

```
=== Difficulty Selection ===


Difficulty:
1. EASY: Smell the flowers.
2. MEDIUM: Hold your ground.
3. HARD: This will make your life hell.
Choose a stat to increase: (1=EASY, 2=MEDIUM, 3=HARD)
> 2
You have selected MEDIUM difficulty.
```



by Zoe McFife

```
1. Create Character (Current: Starjumper)
2. Choose Difficulty (Current: MEDIUM)
3. Start Game
4. Exit
Choose an option (1-4)
> 3
Starting game...
```



```
Battle 1 begins!
Danger Level: HARMLESS

=== Turn 1 ===

----------------------------------------
| Abyssal Marauder                     |
| Health: 35 / 35                      |
| ==================================== |
| D2, I1, S1                           |
| Attack: 14, Defense: 20              |
| (ALIVE)                              |
----------------------------------------
----------------------------------------
| Starjumper                           |
| Health: 182 / 182                    |
| ==================================== |
| Level 1 - D2, I1, S5                 |
| Attack: 18, Defense: 18              |
| Armour: PRISTINE                     |
| (ALIVE)                              |
----------------------------------------
Abyssal Marauder is thinking...

=== Your Turn! ===


== Select an Enemy to Target ==

1. Abyssal Marauder
Select an enemy by entering the corresponding number.
> 1

== Battle Options ==
```

```
Current PP: 0/100
1. Attack
2. Desperate Flurry (5 PP)
3. Defend
4. Use Item
Select an action by entering the corresponding number.
> 1
You chose to Attack!

Abyssal Marauder attacks Starjumper for 14 damage!
Starjumper takes 0 damage!
Starjumper gained 10 PP! (Current PP: 10)
Starjumper attacks Abyssal Marauder for 18 damage!
Abyssal Marauder takes 0 damage!
Press Enter to continue...
```

```
=== Turn 2 ===

----------------------------------------
| Abyssal Marauder                     |
| Health: 35 / 35                      |
| ==================================== |
| D2, I1, S1                           |
| Attack: 14, Defense: 20              |
| (ALIVE)                              |
----------------------------------------
----------------------------------------
| Starjumper                           |
| Health: 182 / 182                    |
| ==================================== |
| Level 1 - D2, I1, S5                 |
| Attack: 18, Defense: 18              |
| Armour: PRISTINE                     |
| (ALIVE)                              |
```

```
---------------------------------------
Abyssal Marauder is thinking...

=== Your Turn! ===


== Select an Enemy to Target ==

1. Abyssal Marauder
Select an enemy by entering the corresponding number.
> 1

== Battle Options ==

Current PP: 10/100
1. Attack
2. Desperate Flurry (5 PP)
3. Defend
4. Use Item
Select an action by entering the corresponding number.
> 3
You chose to Defend!

Starjumper is defending for 4.0 extra defense!
Starjumper gained 10 PP! (Current PP: 20)
Abyssal Marauder attacks Starjumper for 14 damage!
Starjumper takes 0 damage!
Starjumper stopped defending!
Press Enter to continue...
```

```
=== Turn 3 ===

----------------------------------------
| Abyssal Marauder                     |
```

```
| Health: 35 / 35                      |
| ==================================== |
| D2, I1, S1                           |
| Attack: 14, Defense: 20              |
| (ALIVE)                              |
----------------------------------------
----------------------------------------
| Starjumper                           |
| Health: 182 / 182                    |
| ==================================== |
| Level 1 - D2, I1, S5                 |
| Attack: 18, Defense: 18              |
| Armour: PRISTINE                     |
| (ALIVE)                              |
----------------------------------------
Abyssal Marauder is thinking...

=== Your Turn! ===


== Select an Enemy to Target ==

1. Abyssal Marauder
Select an enemy by entering the corresponding number.
> 1

== Battle Options ==

Current PP: 20/100
1. Attack
2. Desperate Flurry (5 PP)
3. Defend
4. Use Item
Select an action by entering the corresponding number.
> 2
You chose to use Desperate Flurry

Abyssal Marauder attacks Starjumper for 14 damage!
Starjumper takes 0 damage!
Desperate Flurry strikes with overwhelming force!
Starjumper uses Desperate Flurry on Abyssal Marauder for 34
damage!
```

```
PP remaining: 15
Abyssal Marauder takes 14 damage!
Press Enter to continue...
```

```
=== Turn 4 ===

----------------------------------------
| Abyssal Marauder                     |
| Health: 21 / 35                      |
| ====================                 |
| D2, I1, S1                           |
| Attack: 14, Defense: 20              |
| (HURT)                               |
----------------------------------------
----------------------------------------
| Starjumper                           |
| Health: 182 / 182                    |
| ==================================== |
| Level 1 - D2, I1, S5                 |
| Attack: 18, Defense: 18              |
| Armour: PRISTINE                     |
| (ALIVE)                              |
----------------------------------------
Abyssal Marauder is thinking...

=== Your Turn! ===


== Select an Enemy to Target ==

1. Abyssal Marauder
Select an enemy by entering the corresponding number.
> 1
```

```
== Battle Options ==

Current PP: 15/100
1. Attack
2. Desperate Flurry (5 PP)
3. Defend
4. Use Item
Select an action by entering the corresponding number.
> 2
You chose to use Desperate Flurry

Abyssal Marauder attacks Starjumper for 14 damage!
Starjumper takes 0 damage!
Desperate Flurry strikes with overwhelming force!
Starjumper uses Desperate Flurry on Abyssal Marauder for 34
damage!
PP remaining: 10
Abyssal Marauder takes 14 damage!
Press Enter to continue...
```

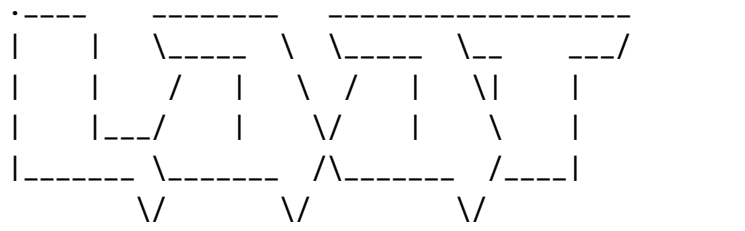
```
=== Turn 5 ===

----------------------------------------
| Abyssal Marauder                     |
| Health: 7 / 35                       |
| ======                               |
| D2, I1, S1                           |
| Attack: 4, Defense: 20               |
| (CRITICALLY_HURT)                    |
----------------------------------------
----------------------------------------
| Starjumper                           |
| Health: 182 / 182                    |
| ==================================== |
```

```
| Level 1 - D2, I1, S5                |
| Attack: 18, Defense: 18             |
| Armour: PRISTINE                    |
| (ALIVE)                             |
---------------------------------------
Abyssal Marauder is thinking...

=== Your Turn! ===


== Select an Enemy to Target ==

1. Abyssal Marauder
Select an enemy by entering the corresponding number.
>
Invalid input. Enter a number between 1 and 1.
> 1

== Battle Options ==

Current PP: 10/100
1. Attack
2. Desperate Flurry (5 PP)
3. Defend
4. Use Item
Select an action by entering the corresponding number.
> 2
You chose to use Desperate Flurry

Abyssal Marauder is defending for 7.0 extra defense!
Desperate Flurry strikes with overwhelming force!
Starjumper uses Desperate Flurry on Abyssal Marauder for 34
damage!
PP remaining: 5
Abyssal Marauder takes 7 damage!
Abyssal Marauder died!
Starjumper gained 50 experience points.
Starjumper leveled up to level 2!
Press Enter to continue...
```

```
Battle 1 has ended!
Press Enter to continue...
```

```
.____    ________   ____________________
|    |   \_____  \  \_____  \__    ___/
|    |    /   |   \  /   |   \|    |
|    |___/    |    \/    |    \    |
|_______ \_______  /\_______  /____|
        \/       \/         \/
You can still loot 4 items. THEY ARE COMING...
Lootable Items:
Carry Capacity: 2/25.0
0: Exit
1: Minor Lifeforce Potion +40.0 HP | Weight: 1.2
2: Spore Spitter +13.0 P DMG +11.0 | Weight: 3.0    Compare: -
1.0 DMG
3: Rundown Vest +8.9 DEF (PRISTINE) | Weight: 3.5    Compare:
+1.9196841757531242 DEF
4: Cargo Clamp +7.0 DEF | Weight: 3.0    Compare: +3.0 DEF
Select an item to loot!
> 3
You looted: Rundown Vest +8.9 DEF (PRISTINE)
```

```
.____    ________   ___________________
|    |   \_____  \  \_____  \__    ___/
|    |    /   |   \  /   |   \|    |
|    |___/    |    \/    |    \    |
|_______ \_______  /\_______  /____|
        \/       \/         \/
You can still loot 3 items. THEY ARE COMING...
Lootable Items:
Carry Capacity: 6/25.0
0: Exit
1: Minor Lifeforce Potion +40.0 HP | Weight: 1.2
2: Spore Spitter +13.0 P DMG +11.0 | Weight: 3.0    Compare: -
1.0 DMG
3: Cargo Clamp +7.0 DEF | Weight: 3.0    Compare: +3.0 DEF
Select an item to loot!
> 3
You looted: Cargo Clamp +7.0 DEF
```



```
You can still loot 2 items. THEY ARE COMING...
Lootable Items:
Carry Capacity: 9/25.0
```

```
0: Exit
1: Minor Lifeforce Potion +40.0 HP | Weight: 1.2
2: Spore Spitter +13.0 P DMG +11.0 | Weight: 3.0    Compare: -
1.0 DMG
Select an item to loot!
> 1
You looted: Minor Lifeforce Potion +40.0 HP
```

```
.____    ________   ___________________
|    |   \_____  \  \_____  \__    ___/
|    |    /   |   \  /   |   \|    |
|    |___/    |    \/    |    \    |
|_______ \_______  /\_______  /____|
        \/       \/         \/
You can still loot 1 items. THEY ARE COMING...
Lootable Items:
Carry Capacity: 10/25.0
0: Exit
1: Spore Spitter +13.0 P DMG +11.0 | Weight: 3.0    Compare: -
1.0 DMG
Select an item to loot!
> 0
```

```
Press Enter to continue...
```

```
  _____  ______  _____ _______
 |  __ \|  ____|/ ____|__   __|
 | |__) | |__  | (___    | |
 |  _  /|  __|  \___ \   | |
 | | \ \| |____ ____) |  | |
 |_|  \_\______|_____/   |_|

----------------------------------------
| Starjumper                           |
| Health: 252 / 252                    |
| ==================================== |
| Level 2 - D2, I1, S5                 |
| Attack: 18, Defense: 18              |
| Armour: PRISTINE                     |
| (ALIVE)                              |
----------------------------------------

== You have survived the battle! ==

1. View Character
2. Use Stat Points (1 available)
3. Equip Item
4. Drop Item
5. Use Items
6. Continue Journey
7. Give up
Select an action by entering the corresponding number.
> 1
```

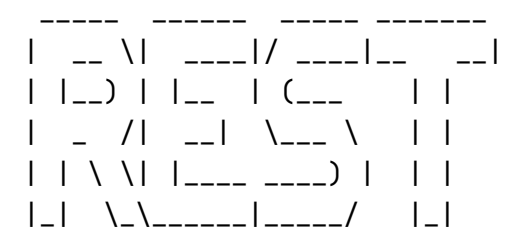
```
=== Information ===
```

```
Name:          Starjumper
Level:         2
Experience:    0/100

=== Stats ===

Health:        252.0/252.0
Defense:             18.0
Attack:        18.2
Strength:            5
Dexterity:           2
Intelligence:        1

=== Equipment ===


== Weapon ==

Name:    Fists
Weight:  0.0
Value:   0.0
Rarity:  LOW
Damage:  14.0
Is Magic:      false
Special: Desperate Flurry
Special Damage:      16.0

== Armor ==

Name:    Clothes
Weight:  1.0
Value:   50.0
Rarity:  LOW
Defense: 7.0
Durability:    675.0 / 675.0 (PRISTINE)
State:   PRISTINE

== Shield ==

Name:    Fists
Weight:  0.0
```

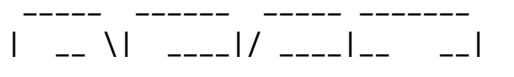
```
Value:  0.0
Rarity: LOW
Defense: 4.0

=== Inventory ===

Inventory:
Weight: 9.9/25.0
1: Refreshing Decoction +38.0 HP | Weight: 1.2
2: Stitching Kit +25.0 HP | Weight: 0.9
3: Cute cat bandaid +25.0 HP | Weight: 0.1
4: Rundown Vest +8.9 DEF (PRISTINE) | Weight: 3.5    Compare:
+1.9 DEF
5: Cargo Clamp +7.0 DEF | Weight: 3.0    Compare: +3.0 DEF
6: Minor Lifeforce Potion +40.0 HP | Weight: 1.2
Press Enter to continue...
```

```
----------------------------------------
| Starjumper                           |
| Health: 252 / 252                    |
| ==================================== |
| Level 2 - D2, I1, S5                 |
| Attack: 18, Defense: 18              |
| Armour: PRISTINE                     |
| (ALIVE)                              |
----------------------------------------

== You have survived the battle! ==
```

```
1. View Character
2. Use Stat Points (1 available)
3. Equip Item
4. Drop Item
5. Use Items
6. Continue Journey
7. Give up
Select an action by entering the corresponding number.
> 2

Allocate your skill points:
1. Strength: 5
2. Dexterity: 2
3. Intelligence: 1
Remaining Points: 1
Choose a stat to increase: (1=STR, 2=DEX, 3=INT)
> 1
```



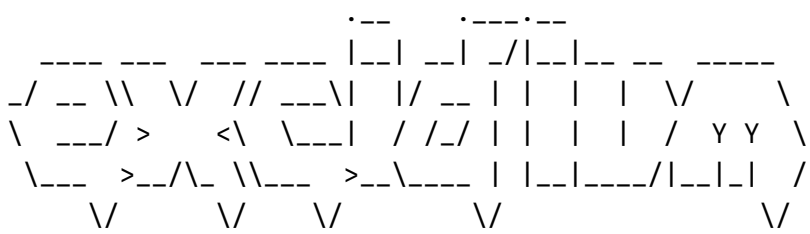
```
| Starjumper                          |
| Health: 270 / 270                   |
| ================================== |
| Level 2 - D2, I1, S6                |
| Attack: 19, Defense: 18             |
| Armour: PRISTINE                    |
| (ALIVE)                             |
---------------------------------------

== You have survived the battle! ==

1. View Character
2. Use Stat Points (0 available)
3. Equip Item
4. Drop Item
5. Use Items
6. Continue Journey
7. Give up
Select an action by entering the corresponding number.
> 3
Inventory:
Weight: 9.9/30.0
1: Refreshing Decoction +38.0 HP | Weight: 1.2
2: Stitching Kit +25.0 HP | Weight: 0.9
3: Cute cat bandaid +25.0 HP | Weight: 0.1
4: Rundown Vest +8.9 DEF (PRISTINE) | Weight: 3.5    Compare:
+1.9 DEF
5: Cargo Clamp +7.0 DEF | Weight: 3.0    Compare: +3.0 DEF
6: Minor Lifeforce Potion +40.0 HP | Weight: 1.2
> 4
Starjumper obtained Clothes!
Starjumper equipped Rundown Vest!
```



```
 | |__) | |__  | (___    | |
 |  _  /|  __|  \___ \   | |
 | | \ \| |____ ____) |  | |
 |_|  \_\______|_____/   |_|

----------------------------------------
| Starjumper                           |
| Health: 270 / 270                    |
| ==================================== |
| Level 2 - D2, I1, S6                 |
| Attack: 19, Defense: 20              |
| Armour: PRISTINE                     |
| (ALIVE)                              |
----------------------------------------

== You have survived the battle! ==

1. View Character
2. Use Stat Points (0 available)
3. Equip Item
4. Drop Item
5. Use Items
6. Continue Journey
7. Give up
Select an action by entering the corresponding number.
> 7
You have chosen to give up. Game Over.
Starjumper has given up.
Starjumper gained 10 PP! (Current PP: 15)
Starjumper attacks Starjumper for 19 damage!
Starjumper takes 19 damage!

Starjumper gained 10 PP! (Current PP: 25)
Starjumper attacks Starjumper for 19 damage!
Starjumper takes 19 damage!

Starjumper gained 10 PP! (Current PP: 35)
Starjumper attacks Starjumper for 19 damage!
Starjumper takes 19 damage!

Starjumper gained 10 PP! (Current PP: 45)
Starjumper attacks Starjumper for 19 damage!
```

```
Starjumper takes 19 damage!

Starjumper gained 10 PP! (Current PP: 55)
Starjumper attacks Starjumper for 19 damage!
Starjumper takes 19 damage!

Starjumper gained 10 PP! (Current PP: 65)
Starjumper attacks Starjumper for 19 damage!
Starjumper takes 19 damage!

Starjumper gained 10 PP! (Current PP: 75)
Starjumper attacks Starjumper for 19 damage!
Starjumper takes 19 damage!

Starjumper gained 10 PP! (Current PP: 85)
Starjumper attacks Starjumper for 19 damage!
Starjumper takes 19 damage!

Starjumper gained 10 PP! (Current PP: 95)
Starjumper attacks Starjumper for 19 damage!
Starjumper takes 19 damage!

Starjumper gained 10 PP! (Current PP: 105)
Starjumper attacks Starjumper for 19 damage!
Starjumper takes 19 damage!

Starjumper gained 10 PP! (Current PP: 115)
Starjumper attacks Starjumper for 6 damage!
Starjumper takes 6 damage!

Starjumper gained 10 PP! (Current PP: 125)
Starjumper attacks Starjumper for 6 damage!
Starjumper takes 6 damage!

Starjumper gained 10 PP! (Current PP: 135)
Starjumper attacks Starjumper for 6 damage!
Starjumper takes 6 damage!

Starjumper gained 0 PP! (Current PP: 135)
Starjumper attacks Starjumper for 6 damage!
Starjumper takes 6 damage!
```

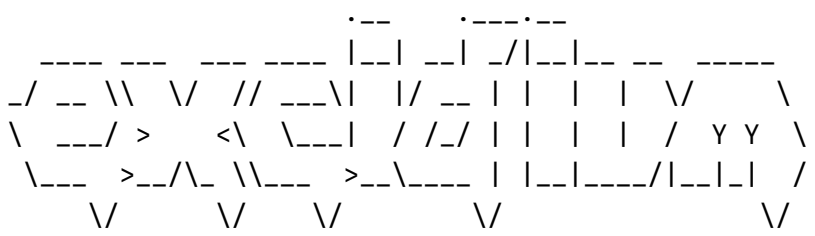
```
Starjumper gained 0 PP! (Current PP: 135)
Starjumper attacks Starjumper for 6 damage!
Starjumper takes 6 damage!

Starjumper gained 0 PP! (Current PP: 135)
Starjumper attacks Starjumper for 6 damage!
Starjumper takes 6 damage!

Starjumper gained 0 PP! (Current PP: 135)
Starjumper attacks Starjumper for 6 damage!
Starjumper takes 6 damage!

Starjumper gained 0 PP! (Current PP: 135)
Starjumper attacks Starjumper for 6 damage!
Starjumper takes 6 damage!

Starjumper gained 0 PP! (Current PP: 135)
Starjumper attacks Starjumper for 6 damage!
Starjumper takes 6 damage!

Starjumper gained 0 PP! (Current PP: 135)
Starjumper attacks Starjumper for 6 damage!
Starjumper takes 6 damage!

Starjumper gained 0 PP! (Current PP: 135)
Starjumper attacks Starjumper for 6 damage!
Starjumper takes 6 damage!

Starjumper gained 0 PP! (Current PP: 135)
Starjumper attacks Starjumper for 6 damage!
Starjumper takes 6 damage!

Starjumper gained 0 PP! (Current PP: 135)
Starjumper attacks Starjumper for 6 damage!
Starjumper takes 6 damage!

Starjumper gained 0 PP! (Current PP: 135)
Starjumper attacks Starjumper for 6 damage!
Starjumper takes 6 damage!
Starjumper died!

Starjumper has given up.
```



by Zoe McFife

```
=== Starjumper has perished in battle! ===

----------------------------------------
| Starjumper                           |
| Health: 0 / 270                      |
|                                      |
| Level 2 - D2, I1, S6                 |
```

```
| Attack: 19, Defense: 19              |
| Armour: SCRATCHED                    |
| (DEAD)                               |
----------------------------------------
Press Enter to continue...
```



by Zoe McFife

1. Create Character
2. Choose Difficulty (Current: MEDIUM)
3. Start Game
4. Exit
Choose an option (1-4)
>